

PPD - Laborator 3 - Documentatie

MPI

MPI - Tip	Tip Matrice	Numar	Nr Threads	Timp Executie (secunde)
Varianta 0 - Secvential	N1=N2=16	Numar1=9999 4444 4444 9999 Numar1=9999 5555 5555 9999	-	0.0006474
	N1=N2=10000	Random	2	0.00566582
	N1=100 N2=100000	Random	-	0.02121346
Varianta 1 - Standard	N1=N2=16	Numar1=9999 4444 4444 9999 Numar1=9999 5555 5555 9999	5	0.00128234
	N1=N2=10000	Random	5	0.0102715
			9	0.01878948
			17	0.03469119
	N1=100 N2=100000	Random	5	0.04132457
			9	0.08736356
			17	0.1865976
Varianta 1.1 - Standard Optimizat	N1=N2=16	Numar1=9999 4444 4444 9999 Numar1=9999 5555 5555 9999	5	0.00149986
	N1=N2=10000	Random	5	0.0093657
			9	0.01758866
			17	0.04137268
	N1=100 N2=100000	Random	5	0.04082034
			9	0.09178774
			17	0.1783005
Varianta 2 - Scatter/Gather	N1=N2=16	Numar1=9999 4444 4444 9999	4	0.00110085

MPI - Tip	Tip Matrice	Numar	Nr Threads	Timp Executie (secunde)
		Numar1=9999 5555 5555 9999		
	N1=N2=10000	Random	4	0.00404575
			8	0.00455653
			16	0.00553764
	N1=100 N2=100000	Random	4	0.01675516
			8	0.0161078
			16	0.01794027
Varianta 3 - Async	N1=N2=16	Numar1=9999 4444 4444 9999 Numar1=9999 5555 5555 9999	5	0.00157691
	N1=N2=10000	Random	5	0.00979875
			9	0.0190411
			17	0.03575964
	N1=100 N2=100000	Random	5	0.04796005
			9	0.08534484
			17	0.1881649

Varianta 1: Standard Neoptimizată (StandardCalculation.cpp)

- Model: Manager-Worker (Rank 0 este managerul, iar P1...P-1 sunt workerii).
- **Procesul 0 (Manager):** -Distribuție: Într-o primă buclă, P0 citește secvențial câte un segment de date din fișiere și îl trimite fiecărui worker (P1, P2,... P-1).
 - Colectare: Într-o a doua buclă, P0 așteaptă segmentul de rezultat de la fiecare worker, în ordine, și îl scrie imediat în fișierul de ieșire.
 - Carry Final: La final, P0 așteaptă un ultim carry (transport) de la ultimul worker (P-1). Dacă acest carry este 1, îl scrie la sfârșitul fișierului, indicând creșterea dimensiunii rezultatului.
- **Procesele Worker (P1...P-1):**
 1. Primesc segmentul de cifre de la P0.

2. Așteaptă Carry: Așteaptă un carry de la procesul anterior (rank-1). Excepție: Procesul P1 nu primește nimic și pornește cu carry = 0
3. Calcul: Numai după ce au primit carry, procesele își calculează suma locală (sum()).
4. Aplicare: Aplică carry-ul primit peste suma locală (folosind passCarry) Această operație actualizează și carry-ul pe care worker-ul îl va trimite mai departe.
5. Trimitere: Fiecare worker trimite propriul carry final către procesul următor (rank+1) și trimite segmentul său de **rezultat** final înapoi la P0.

Varianta 1.1: Standard Optimizată(StandardCalculation.cpp)

- Model: Manager-Worker (Rank 0 este managerul, iar P1...P-1 sunt workerii).
- **Procesul 0 (Manager):**
 - Distribuție: Într-o primă buclă, P0 citește secvențial câte un segment de date din fișiere și îl trimite fiecărui worker (P1, P2,... P-1).
 - Colectare: Într-o a doua buclă, P0 așteaptă segmentul de rezultat de la fiecare worker, în ordine, și îl scrie imediat în fișierul de ieșire.
 - Carry Final: La final, P0 așteaptă un ultim carry (transport) de la ultimul worker (P-1). Dacă acest carry este 1, îl scrie la sfârșitul fișierului, indicând creșterea dimensiunii rezultatului.
- **Procesele Worker (P1...P-1):**
 1. Primesc segmentul de cifre de la P0.
 2. Calcul (Imediat): Procesele își calculează imediat suma locală (sum()), fără a aștepta carry-ul de la procesul anterior.
 3. Așteaptă Carry: După ce calculul local este gata, așteaptă un carry de la procesul anterior (rank-1). Excepție: Procesul P1 nu primește nimic și pornește cu carry = 0.
 4. Aplicare: Aplică carry-ul primit peste suma locală (folosind passCarry). Această operație actualizează și carry-ul pe care worker-ul îl va trimite mai departe.
 5. Trimitere: Fiecare worker trimite propriul carry final către procesul următor (rank+1) și trimite segmentul său de rezultat final înapoi la P0.

Analiza Variantei 1 vs. Varianta 1.1 (Optimizare)

Da, varianta optimizată este posibilă. Diferența dintre acestea se află în logica worker-ului: la prima variantă (V1), worker-ul este blocat pentru că **așteaptă** carry-ul, în schimb, cealaltă variantă (V1.1) **trece** deja la calcularea sumei.

În majoritatea timpilor, mai ales când numărul de procese nu este unul foarte mare, varianta **optimizată câștigă**. Odată cu creșterea numărului de procese, timpii cresc pentru ambele variante din cauză că timpul total de așteptare al tuturor carry-urilor crește direct proporțional cu numărul de procese.

Varianta 2: Comunicare Colectivă (Scatter / Gather ScatterCalculation.cpp)

- Model: Colectiv. Procesul 0 participă activ la calcul, alături de celelalte procese.
- **Procesul 0 (Pregătire și Finalizare):**
 - Citire și Padding: Procesul 0 citește integral ambele numere în memorie. Calculează o dimensiune totală (totalSize) care să fie divizibilă cu numărul de procese (P), adăugând zerouri (padding) la sfârșitul numerelor dacă este necesar.
 - Finalizare: După ce primește rezultatele de la toate procesele, P0 scrie rezultatul final în fișier (având grijă să scrie doar cifrele originale, fără padding-ul adăugat) și așteaptă un ultim carry de la procesul P-1, pe care îl scrie la final, dacă > 0.
- **Toate Procesele (P0...P-1):**
 1. Distribuție: Toate procesele, inclusiv P0, apelează MPI_Scatter de două ori pentru a primi segmentul lor de cifre din primul număr și din al doilea număr. Procesul P0 citește întreg numărul;
 2. Calcul: Fiecare proces își calculează imediat suma locală (sum()).
 3. Pipeline Carry: Se execută un pipeline de transport similar cu cel din varianta optimizată:
 - Procesele cu rank > 0 așteaptă (MPI_Recv) un carry de la procesul anterior (rank-1).
 - Fiecare proces aplică carry-ul primit peste suma sa locală (passCarry).
 - Procesele cu rank < P-1 trimit (MPI_Send) propriul carry final către procesul următor (rank+1).
 4. Colectare: Toate procesele apelează MPI_Gather pentru a trimite segmentele lor de rezultat înapoi la P0, care le assemblează în bufferul final.

Varianta 3: Comunicare Asincronă (AsyncCalculation.cpp)

- Model: Manager-Worker (Rank 0 este managerul, iar P1...P-1 sunt workerii).
- **Procesul 0 (Manager):**
 - Distribuție Asincronă: Într-o buclă, P0 citește segmentele de date și le trimite imediat folosind MPI_Isend. Toate cererile de trimitere sunt salvate.
 - Colectare Asincronă: După ce a inițiat toate trimiterile, P0 postează imediat toate cererile de primire (MPI_Irecv) pentru segmentele de rezultat de la workeri și pentru carry-ul final de la P-1.
 - Sincronizare: P0 apelează MPI_Waitall o dată pentru a aștepta finalizarea tuturor trimiterilor și încă o dată pentru a aștepta finalizarea tuturor primirilor. Aceasta îi permite să primească rezultatele în orice ordine sosesc.
 - Scriere: După ce MPI_Waitall pentru primiri se termină, P0 scrie rezultatul asamblat în fișier.
- **Procesele Worker (P1...P-1):**

1. Postare Primiri: Fiecare worker postează `MPI_Irecv` pentru cele două segmente de cifre (de la P0) și, dacă `rank > 1`, un `MPI_Irecv` pentru carry-ul de la rank-1.
2. Așteptare Generală: Worker-ul apelează `MPI_Waitall` pentru *toate* cererile de primire. Acesta este punctul cheie: procesul așteaptă ca toate datele necesare să sosească, indiferent de ordinea lor (de exemplu, carry-ul de la P2 poate sosi înainte sau după datele de la P0).
3. Calcul: Odată ce `Waitall` se termină, worker-ul are toate datele și execută `sum()` și `passCarry()`.
4. Trimitere Asincronă: Worker-ul trimite rezultatul și carry-ul său folosind `MPI_Isend`.
5. Sincronizare Finală: Apelează `MPI_Waitall` pentru cererile de trimitere, pentru a se asigura că datele au fost trimise înainte de a elibera bufferele și a ieși.